Storage Class, Server Access Logging and Policy Generator

Class no:7

What is a Storage Class in AWS S3?

- A **storage class** in Amazon S3 specifies the **cost, availability, durability, and retrieval time** of objects stored in buckets.(as they can be applied only on object level)
- It allows you to optimize storage costs by choosing the right class based on **data access patterns** (frequent, infrequent, archival). They are applicable for both private and public buckets.

Purpose of Storage Classes

- **Cost optimization**: Pay less for data that is rarely accessed.
- **Performance tuning**: Ensure fast retrieval for frequently accessed data.
- **Data lifecycle management**: Move data automatically between classes as access patterns change.
- **Compliance & durability**: Meet business requirements for availability, redundancy, and geographic isolation.

Types of AWS S3 Storage Classes and Their Purpose

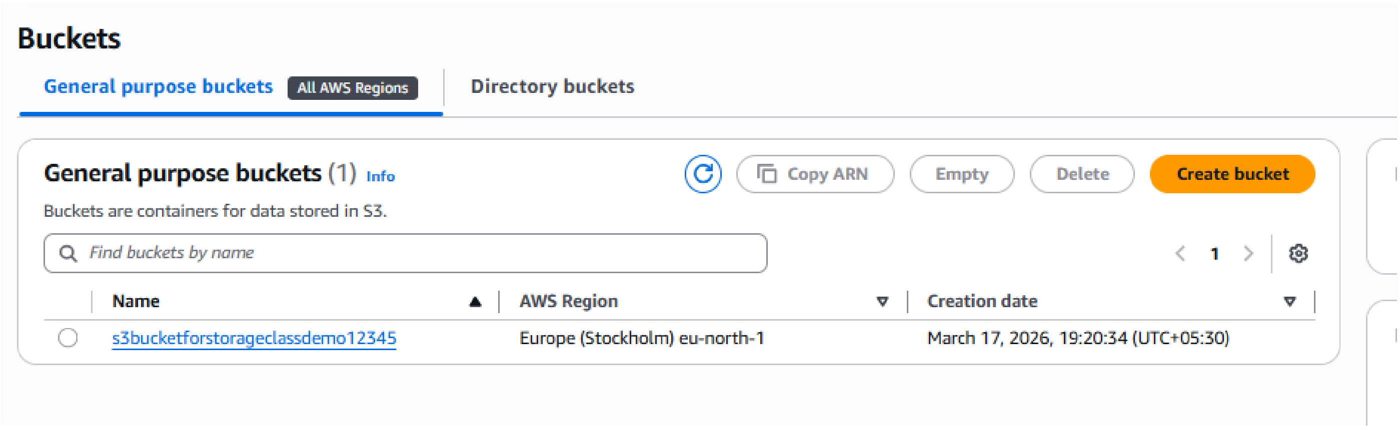
Storage Class	**Durability & Availability**	**Cost**	**Purpose**
S3 Standard	99.999999999% durability, high availability	Higher	For frequently accessed data (websites, apps, analytics).
S3 Express One Zone	High performance, single AZ	Lower than Standard	Ultra-low latency for frequently accessed data in one zone.
S3 Intelligent-Tiering	Same durability as Standard	Variable	Automatically moves data between frequent/infrequent tiers based on usage. Best for unknown or changing access patterns.
S3 Standard-IA (Infrequent Access)	High durability, lower availability	Lower	For data accessed less often but needs quick retrieval (backups, disaster recovery).
S3 One Zone-IA	Stored in one AZ only	Lower	For infrequently accessed data that can be re-created if lost.
S3 Glacier Instant Retrieval	Millisecond retrieval	Very low	For archival data that still needs occasional instant access.
S3 Glacier Flexible Retrieval	Minutes to hours retrieval	Very low	For archives where retrieval speed is flexible.
S3 Glacier Deep Archive	Hours retrieval	Lowest	For long-term archival (regulatory, compliance, rarely accessed data).

Key Points

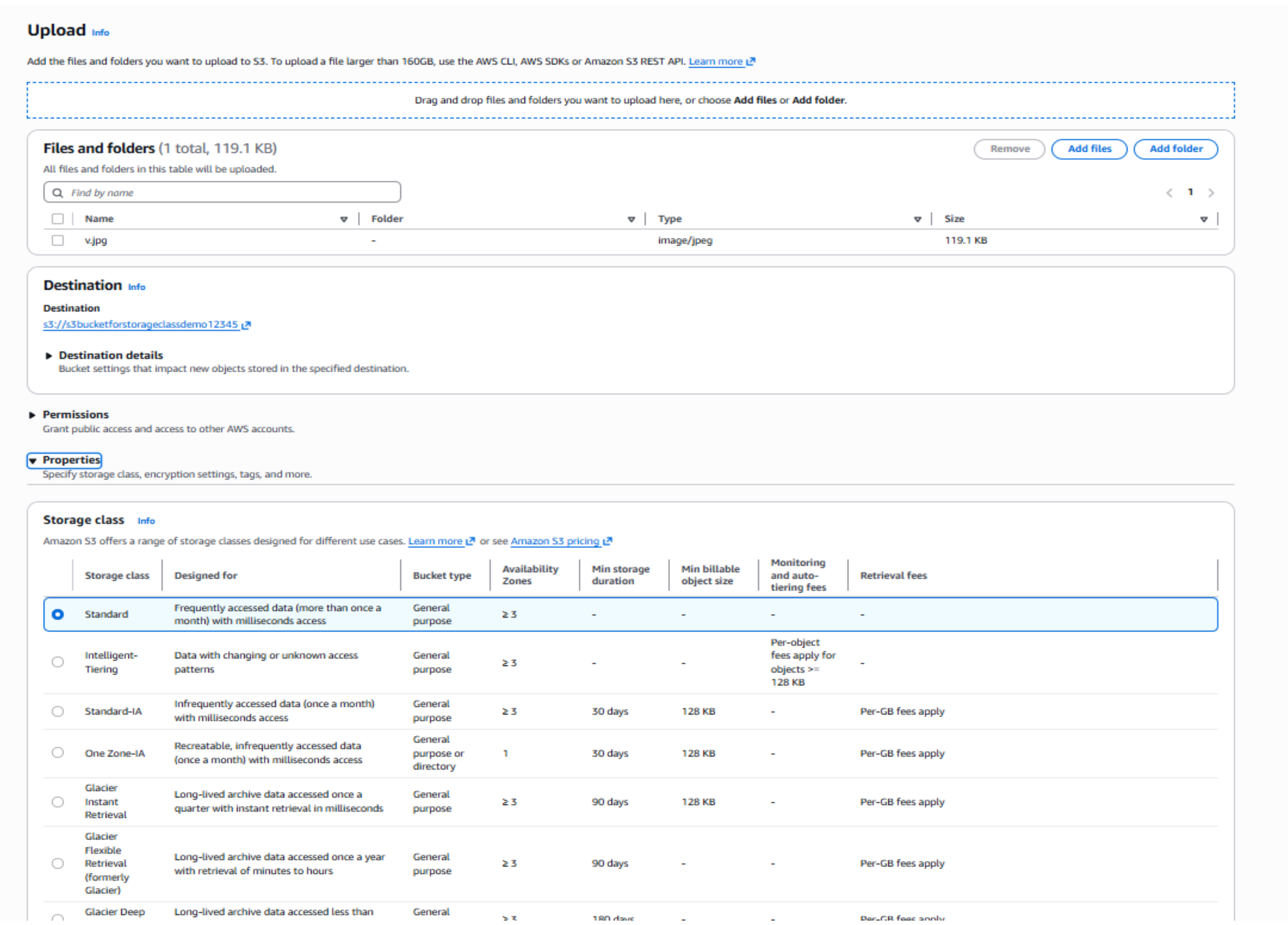
- **Standard** → Best for hot data (frequent use).
- **Intelligent-Tiering** → Best when access patterns are unpredictable.
- **IA classes** → Balance cost and speed for backups or rarely used files.
- **Glacier classes** → Lowest-cost archival storage, retrieval times vary.
- **Express One Zone** → High-speed, single-zone option for performance-sensitive workloads.

Hands on: Creating a bucket and implementing the storage class on object.

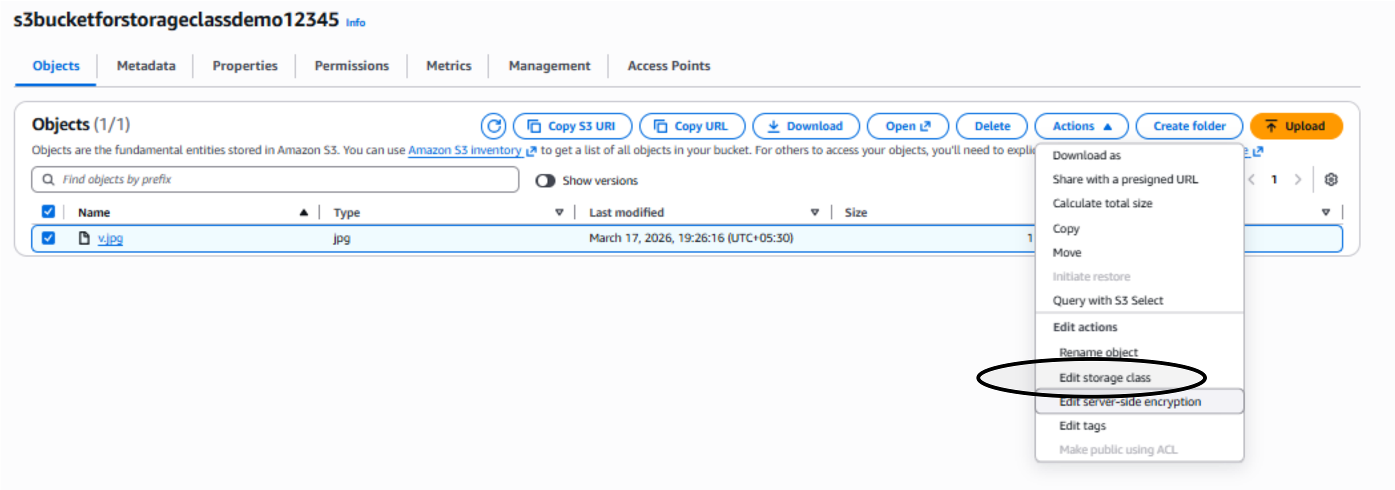
Step no:1 Create a bucket : **s3bucketforstorageclassdemo12345** (it can be either private or public).Here we are creating a private bucket.



Step no:2 Now try to add objects in the bucket. In the page we can see the **properties** section, click on that we can decide the type of storage class depends on the object, how often we are going to access it. Then choose **upload.**



Also, we can change the storage class of an object after uploading it. Click on the object, choose ***actions*** where we can see the **Edit storage class option**



Lifecycle rule in S3: (Storage class setup)

- A **Lifecycle rule** in Amazon S3 is a policy that automatically manages the **transition** and **expiration** of objects in a bucket.
- It defines actions to move objects between **storage classes** or delete them after a certain period.

Purpose of Lifecycle Rules

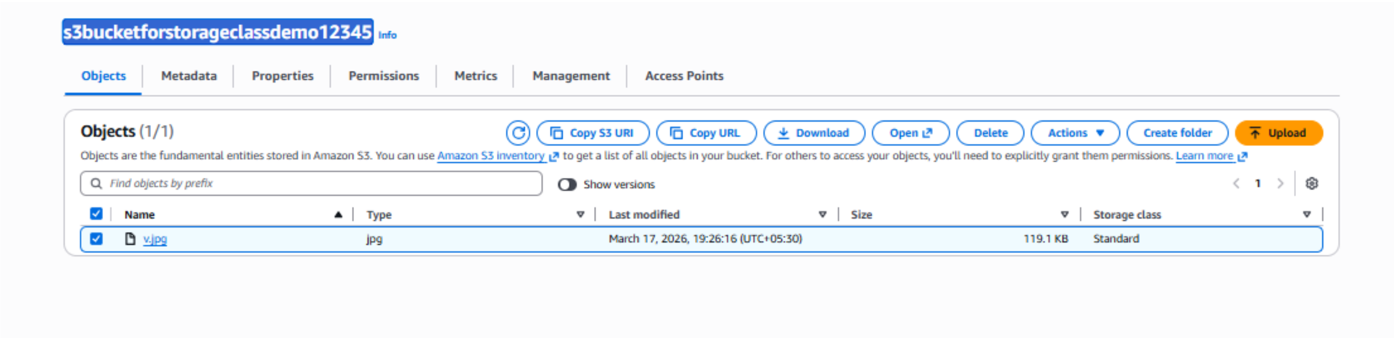
- **Cost optimization**: Move data from expensive storage classes (like Standard) to cheaper ones (like Glacier) when it becomes less frequently accessed.
- **Automation**: Reduce manual effort by automatically handling data transitions and deletions.
- **Data management**: Ensure compliance with retention policies by expiring objects after a set time.
- **Efficiency**: Keep buckets clean and avoid unnecessary storage costs.

Key Actions in Lifecycle Rules

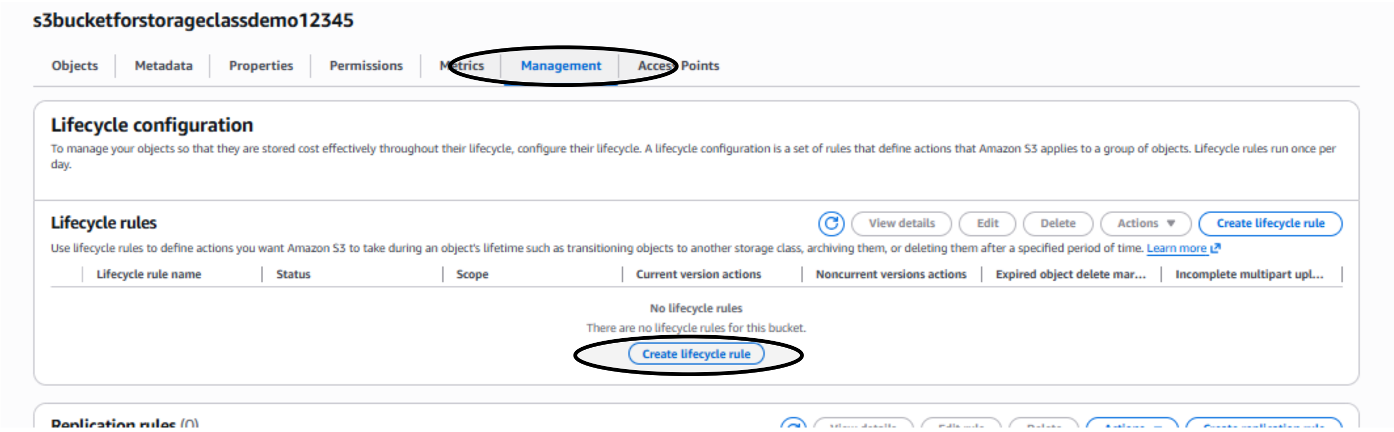
1. **Transition Actions**
   - Move objects to another storage class after a specified number of days.
   - Example: Move logs from **S3 Standard** → **S3 Standard-IA** after 30 days → **S3 Glacier Deep Archive** after 1 year.
2. **Expiration Actions**
   - Permanently delete objects after a defined period.
   - Example: Delete temporary files after 7 days.
3. **Abort Incomplete Multipart Uploads**
   - Automatically clean up incomplete uploads after a set time to save costs.

Hands on for lifecycle rule set up in our created S3 bucket.

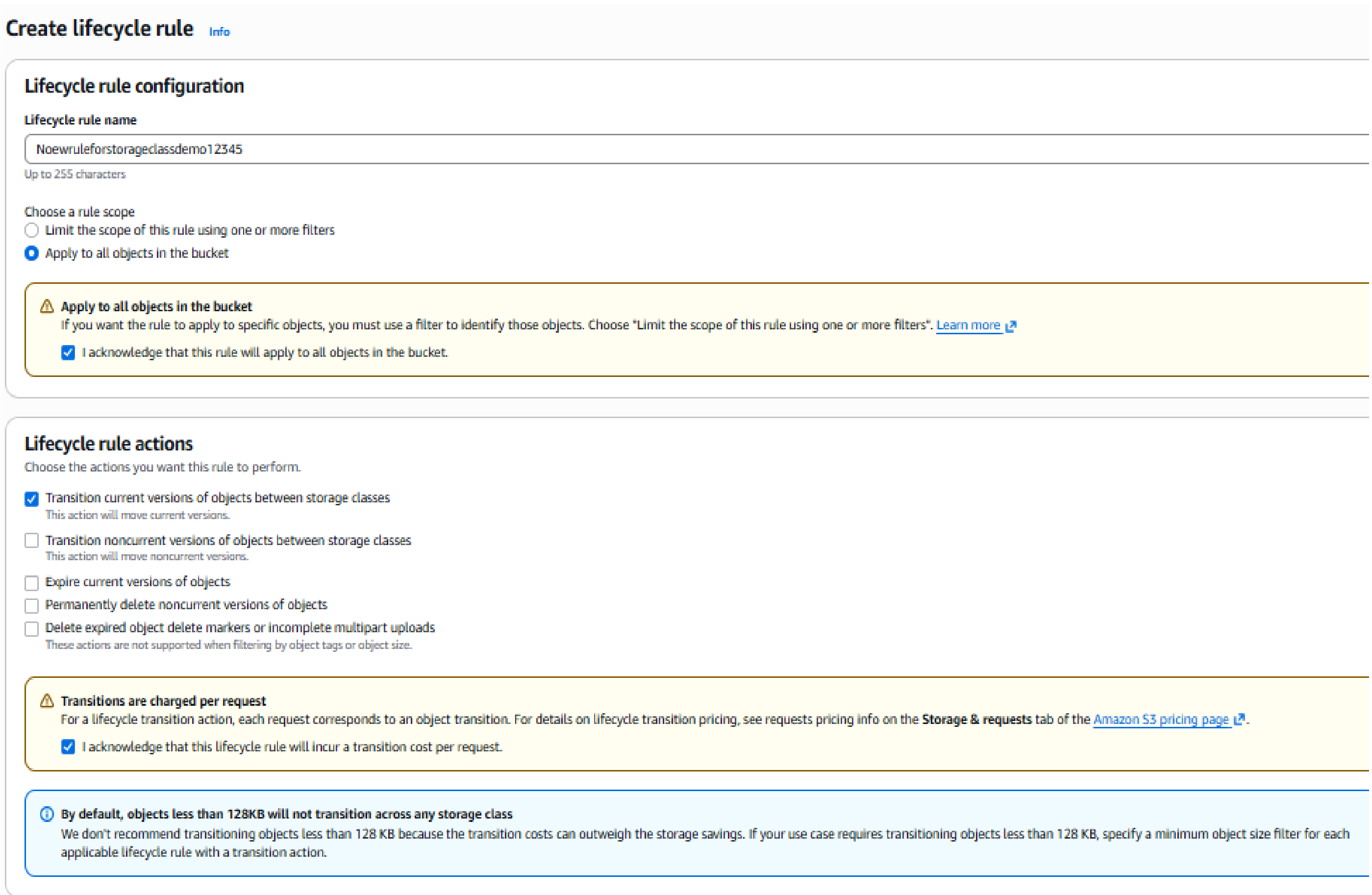
Step no:1 Open our bucket (s3bucketforstorageclassdemo12345) -> move to the object.



Step no:2 Click on the **management** tab on the object page and go to **create lifecycle role** click on it.

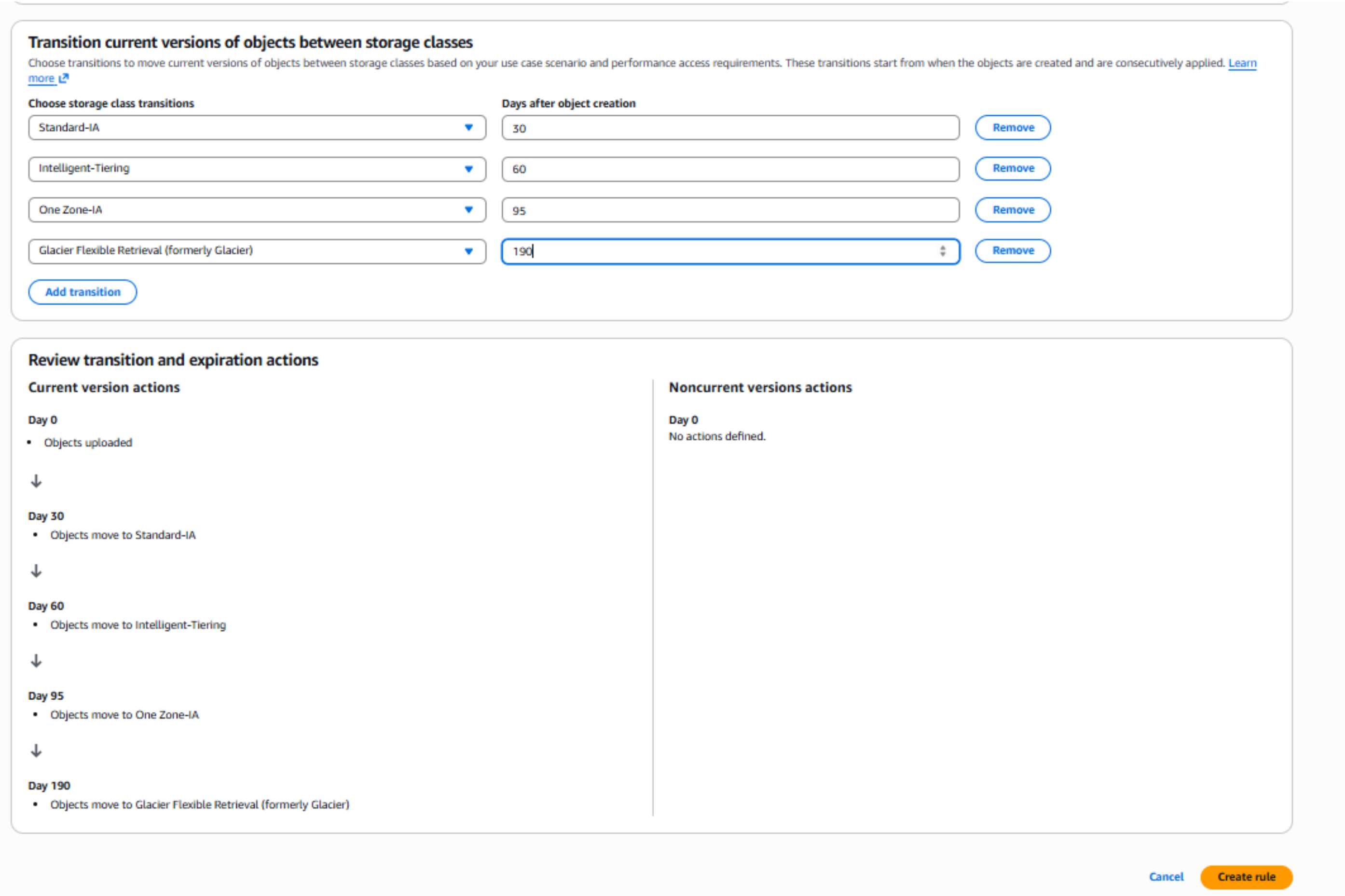


Step no:3 Provide a rule name, enable apply to all objects, enable transition for current versions, acknowledge it.



Now we will configure the **Transition current versions of objects between storage classes.** This is on the same page. Click create rule. It will automate the process.

Note: always check and enter the minimum days for the storage class objects.



Server access Logging:

- Server Access Logging is a feature in Amazon S3 that records detailed information about requests made to your S3 bucket.
- It generates log files that capture details such as the requester, bucket name, request time, action performed, error codes, and more.
- Logs are stored in a designated S3 bucket for later analysis.

Purpose of Server Access Logging

- Audit & Compliance: Track who accessed data and when, useful for security reviews.
- Security Monitoring: Detect unauthorized or suspicious access attempts.
- Usage Analysis: Understand access patterns, frequency, and performance.
- Cost Optimization: Identify unnecessary requests or inefficient data access.
- Troubleshooting: Diagnose issues related to permissions or failed requests.

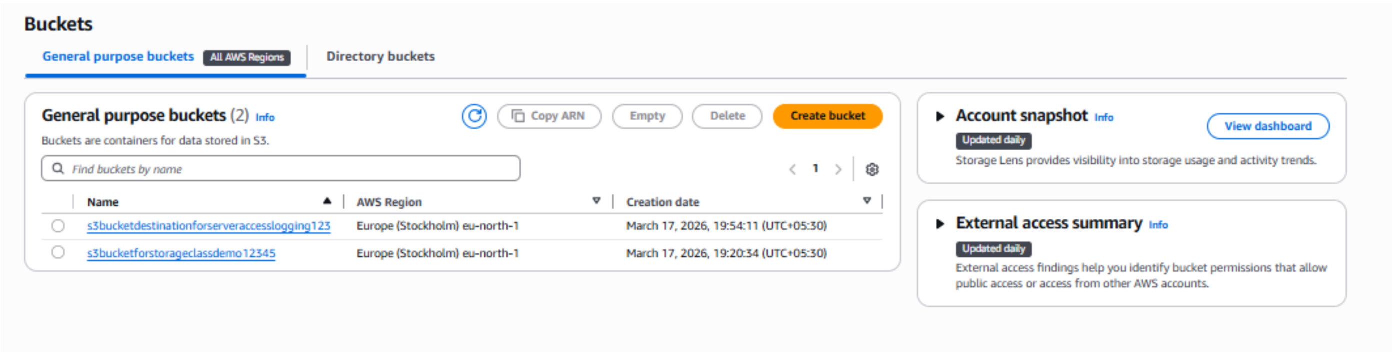
📒 Key Aspects of Server Access Logging

Log Destination and Log Content

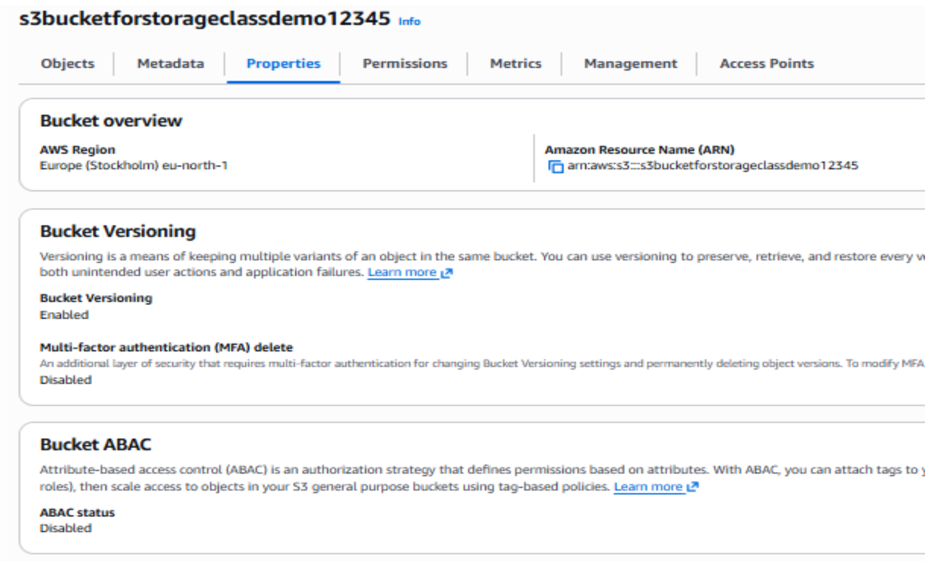
- Logs are delivered to a target S3 bucket (must be in the same AWS region).
- Best practice: use a separate bucket for logs to avoid clutter.

<u>Hands on for Creating a server access logging:</u>

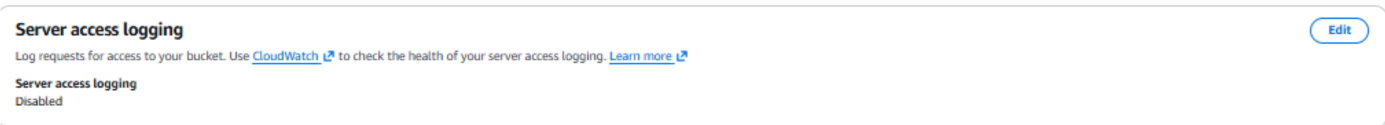
Step no:1 Create two buckets. One is our bucket we want to monitor (source:s3bucketforstorageclassdemo12345) and the other (destination: s3bucketdestinationforserveraccesslogging123) is where we store the logs of the source bucket.



Step no:2 Open our source bucket we want to monitor. Click the properties tab.



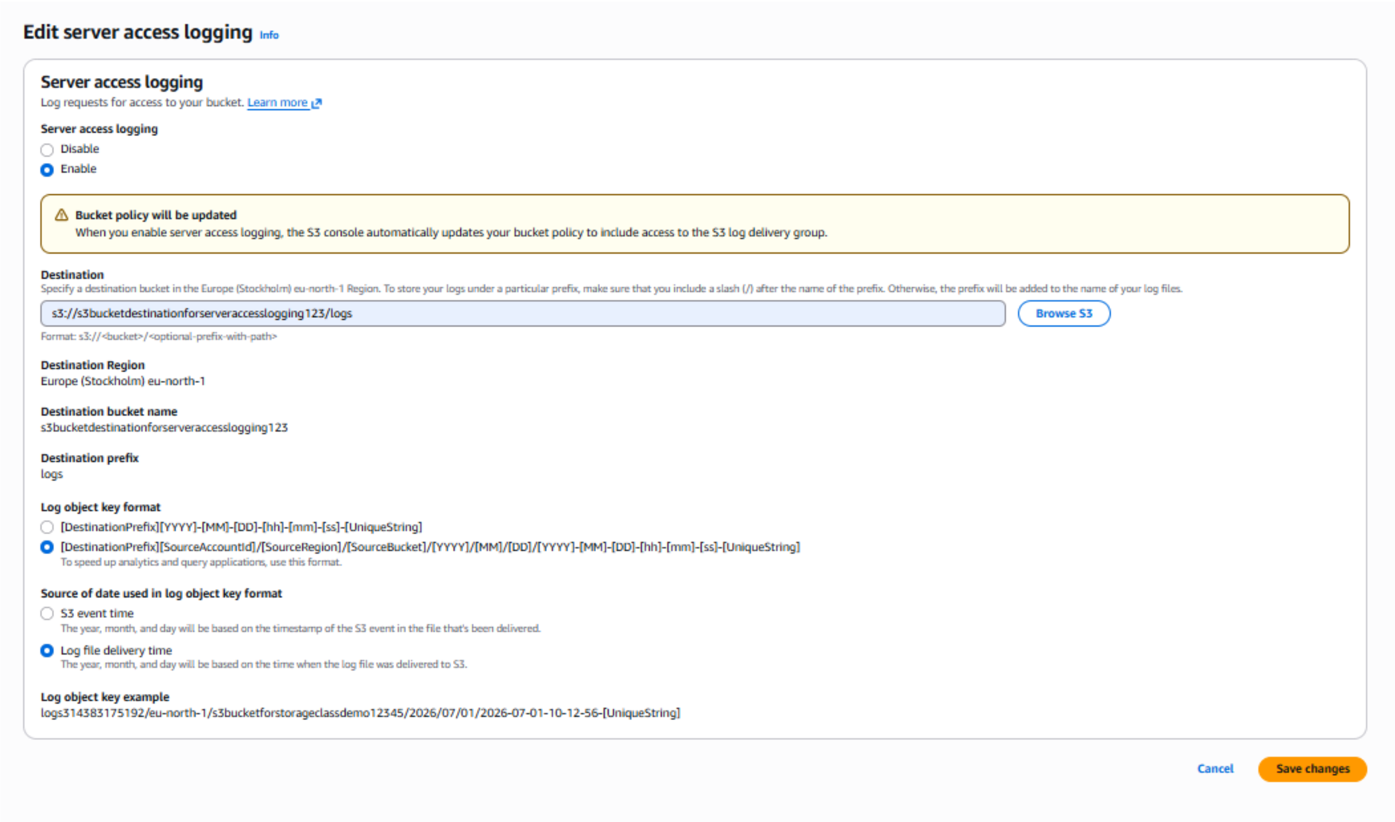
Scroll down. We can see the **server access logging** option. Click **edit.**



Step no:3 Enable Server access login.

(i) Provide the destination bucket name, followed by the suffix /logs to easily identify as log files.
(ii) We can choose the desired format based on that it will show an example below.
(iii) Save changes.

Wait at least 30 minutes to see the logs in the destination bucket we have provided for the changes in the source bucket.



Policy Generator:

- A **web-based tool** that assists in creating **access control policies** for AWS resources.
- It generates **JSON policy documents** that define what actions are allowed or denied on specific AWS services.
- Supports multiple policy types: **IAM policies, S3 bucket policies, SNS topic policies, SQS queue policies, VPC endpoint policies**, etc.

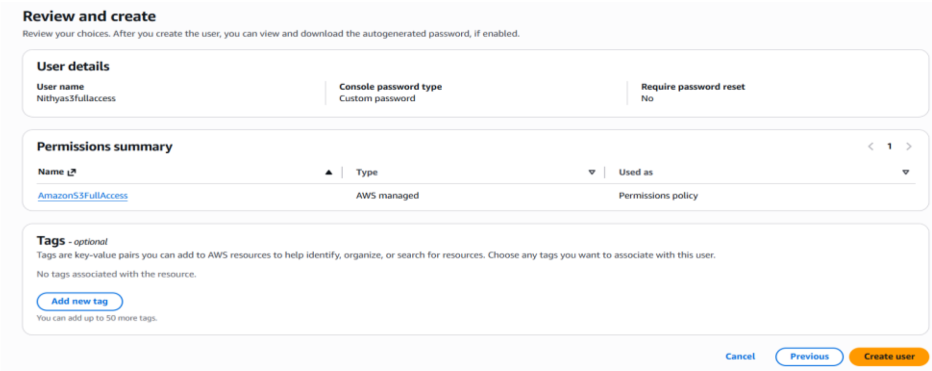
Purpose of the Policy Generator

- **Ease of Use**: Helps users who are not familiar with JSON syntax create valid policies.
- **Consistency**: Ensures policies follow AWS best practices and correct structure.
- **Security**: Allows fine-grained control over access permissions, reducing misconfigurations.
- **Efficiency**: Saves time compared to writing policies manually.

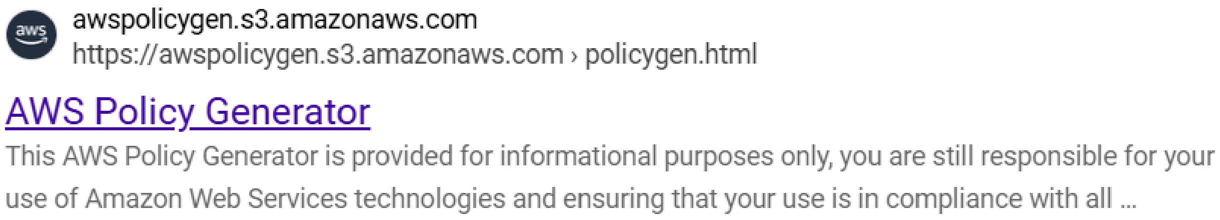
Hands on: Creating a policy for denying an IAM user to access a specific s3 bucket, even though the user has S3 full access policy.

(Note: The user cannot access the specific bucket alone, except that rest of the backets can be accessed by the user.)

Step no:1 Create an IAM user with S3 full access policy.

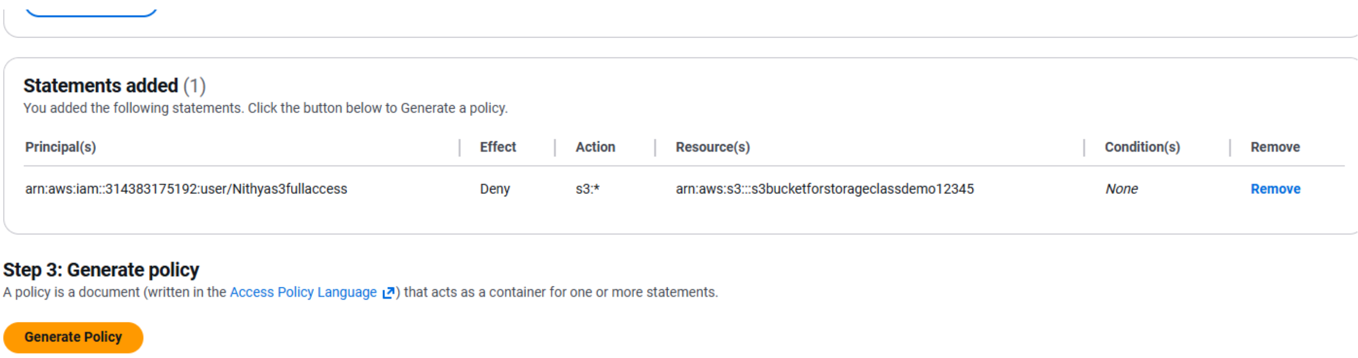


Step no:2 Search AWS policy generator for S3 in the browser and open it.



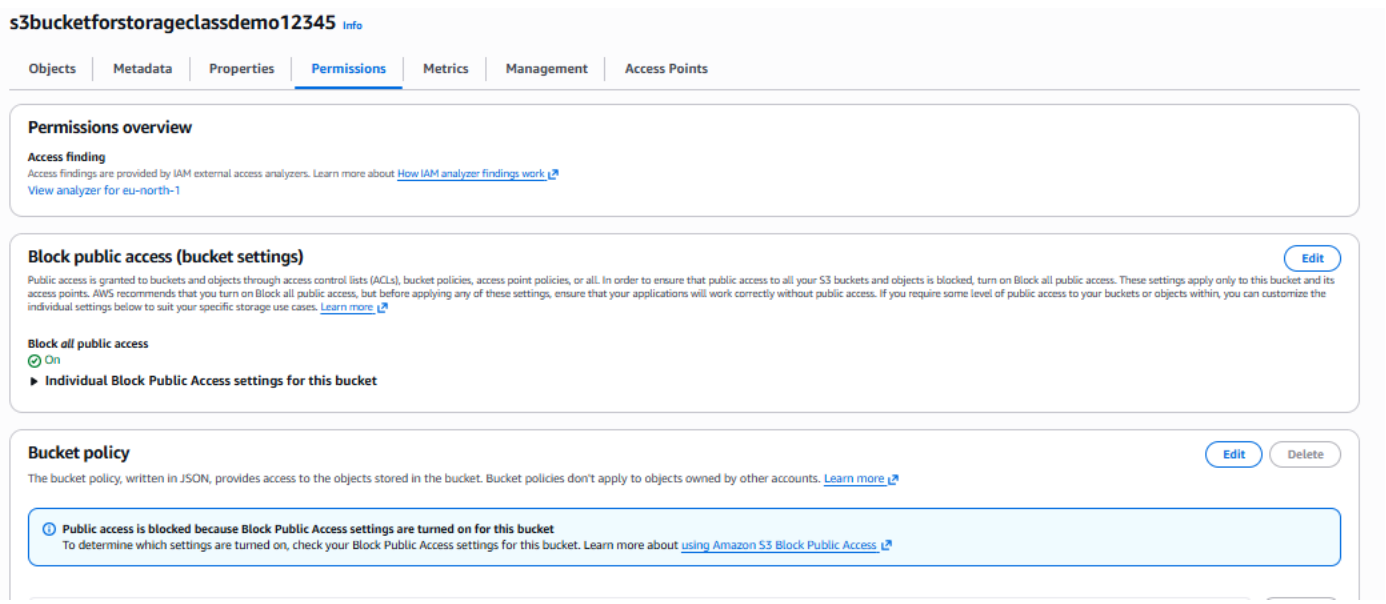
Step no:4 Follow the configurations,

- Type of Policy: S3 Bucket Policy
- Deny
- ARN of the User
- Enable All actions
- ARN of the bucket
- Add statement
- Generate policy

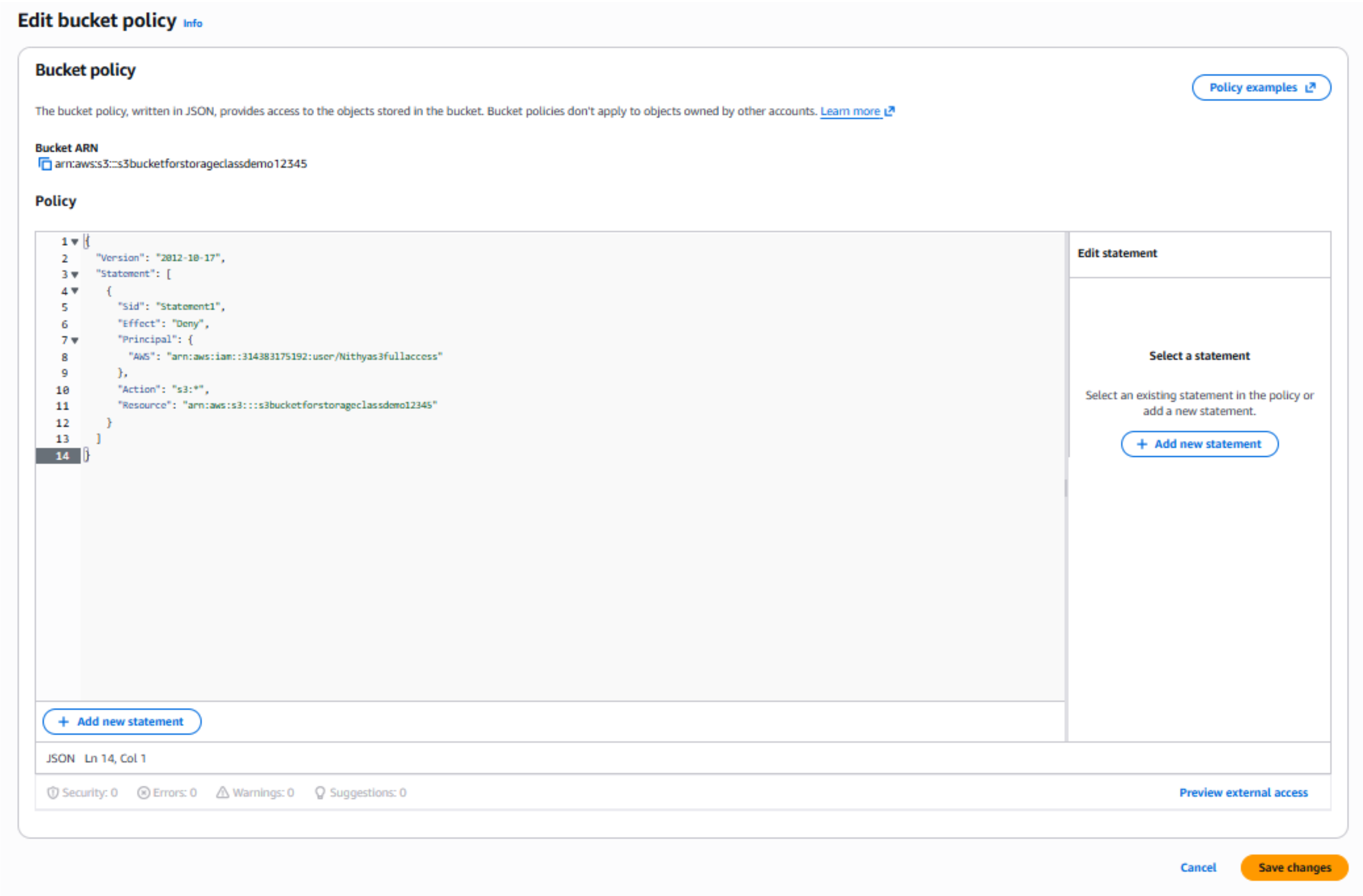


Copy the policy contents.

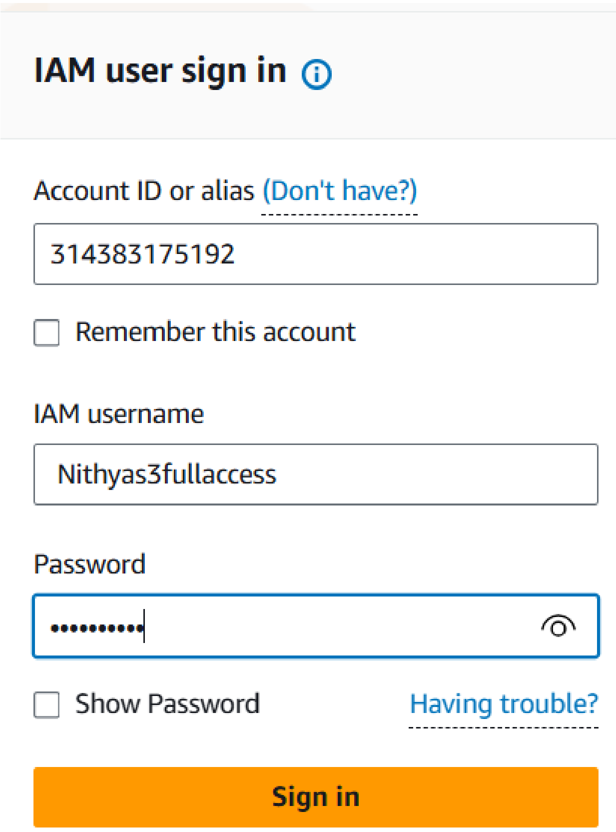
Step no:3 Open our bucket in the console, move to permissions. We can see the bucket policy option. Choose edit.



Step no:4 Now paste the contents of the policy generator. It will be in json format. Save changes.



Step no:5 To check: Open a private window and login as the IAM user who cannot access this bucket.



Then try to access the specific bucket. It will show access denied. It is because we have enabled all actions to be denied during the policy generation.